

ASTEROIDS GAME: C DİLİ VE SDL2 KÜTÜPHANESİ İLE 2D MODÜLER UZAY HAYATTA KALMA OYUNU

**KOCAELİ ÜNİVERSİTESİ
YAZILIM MÜHENDİSLİĞİ BÖLÜMÜ**

DÖNEM PROJESİ RAPORU

**Hazırlayan:
Harun Bozkurt
240229084**

2026

İçindekiler

1.Giriş	3
2.Kullanılan Teknolojiler	3
3.Oyun Mekanikleri	4
4. Yazılım Mimarisi	5
5. Algoritmalar ve Oyun Mekaniği	6
6. Karşılaşılan Zorluklar.....	7
7. Ekran Görüntüleri	9
8. Sonuç	10
9. Kaynakça.....	11

1.Giriş

Bu projenin temel amacı, C programlama dili kullanılarak gerçek zamanlı çalışan bir 2D oyun geliştirmek ve oyun programlama alanındaki temel mekanikleri uygulamalı olarak öğrenmektir.

SDL2 kütüphanesi kullanılarak grafik çizimi, kullanıcı girdileri, ses yönetimi ve çarpışma kontrolü gibi sistemlerin geliştirilmesi hedeflenmiştir. Bunun yanında modüler programlama mantığının uygulanması, oyun içi fizik mekaniklerinin oluşturulması ve performans açısından düzenli bir oyun döngüsü kurulması amaçlanmıştır.

Geliştirilen oyun sayesinde hem teknik bilgi kazanılması hem de oyun geliştirme süreçlerinin deneyimlenmesi hedeflenmiştir.

2.Kullanılan Teknolojiler

Bu projenin geliştirme sürecinde oyun programlama ve multimedya işlemleri için çeşitli teknolojiler ve yardımcı kütüphaneler kullanılmıştır.

- **C Programlama Dili:** Oyunun temel yapısı, oyun döngüsü ve algoritmaları C dili kullanılarak geliştirilmiştir.
- **SDL2:** Pencere oluşturma, grafik çizimleri, kullanıcı girdileri ve temel oyun yönetimi işlemleri için kullanılmıştır.
- **SDL2_image:** PNG formatındaki görsellerin ve oyun içi dokuların işlenmesi amacıyla kullanılmıştır.
- **SDL2_mixer:** Patlama, lazer ve arka plan müzikleri gibi ses efektlerinin yönetimi için kullanılmıştır.
- **SDL2_ttf:** Oyun içi skor, can bilgisi ve menü yazılarının ekrana basılması amacıyla kullanılmıştır.
- **Visual Studio Code:** Projenin geliştirme sürecinde kullanılan kod editörüdür.

3.Oyun Mekanikleri

Oyunun akıcı ve tekdüze olmaması için tamamen gerçek zamanlı hesaplamalara dayalı mekanikler kurulmuştur. Kodlanan ana sistemleri şu şekilde maddelere ayırabiliriz:

- **Gemi Hareketi ve Momentum:** Uzay gemisinin hareketinde gerçekçi bir uzay boşluğu hissiyatı oluşturulmuştur. Gemi yön tuşlarıyla doğrudan hareket etmez; önce kendi ekseninde döndürülerek hedef açı belirlenir ve ileri yönlü itki uygulandığında o açıda hızlanır. İtki kesildiğinde gemi anında durmayarak mevcut hızıyla kaymaya devam eder. Bu dönüş ve ivmelenme mekaniklerinin hesaplanmasında C dilinin matematik kütüphanesindeki sinüs ve kosinüs fonksiyonları kullanılmıştır.
- **Meteor Sistemi:** Meteorlar ekranın dış sınırlarında farklı boyut, hız ve yönlerde rastgele üretilmektedir. Büyük bir meteor vurulduğunda tamamen yok olmak yerine, ikiye bölünerek daha küçük ve hızlı iki yavru meteor halinde farklı açılara savrulur. Bu durum oyunun zorluk seviyesini artırmaktadır.
- **Çarpışma Kontrolü:** Mermilerin meteorlara veya meteorların gemiye çarpma durumu, ana oyun döngüsü içerisinde denetlenmektedir. Nesnelerin merkez noktaları ve yarıçap değerleri hesaplanarak daire tabanlı bir çarpışma mantığı uygulanmıştır. Bir çarpışma gerçekleştiğinde ilgili objeler bellekten temizlenmekte ve oyuncunun can değeri güncellenmektedir.
- **Güçlendirmeler ve Süre Takibi:** Oynanışı hareketlendirmek adına uzay boşluğunda belirli aralıklarla rastgele güçlendirme kutuları belirmektedir. Oyuncu bu kutuları topladığında gemiye geçici bir hasar kalkanı veya aynı anda üç mermi ateşleyebilen çoklu lazer yeteneği kazandırılmaktadır. Bu yeteneklerin aktif kalma süreleri arka planda zamanlayıcı kullanılarak takip edilmektedir.

4. Yazılım Mimarisi

Projenin kaynak kodu, tüm işlemlerin tek bir dosyada toplandığı karmaşık bir yapıdan ziyade, sürdürülebilir ve okunabilir bir mimari olan modüler programlama prensiplerine göre tasarlanmıştır. Oyunun farklı işlevleri kendi içinde bağımsız .c ve .h dosyalarına ayrılarak yönetilmiştir. Projeyi oluşturan temel modüller ve içerikleri aşağıdaki gibidir:

- **main.c:** Oyunun başlangıç ayarlarının yapıldığı, SDL2 pencerelerinin oluşturulduğu ve oyunun sürekli akmasını sağlayan ana döngünün çalıştığı merkez dosyadır. Tüm dokular (texture), ses dosyaları, oyun durumları ve zorluk sayaçları burada tanımlanmış olup; pencerenin boyanması ve oyunun sıfırlanması gibi global işlemler buradan yönetilir.
- **gemiVeMermi.c:** Uzay gemisinin ve mermilerin yaşam döngüsünü kontrol eden modüldür. Geminin yaratılması, hareket hesaplamaları ve geminin ekran sınırları içinde kalmasını sağlayan kontroller burada işlenir. Aynı zamanda oyuncunun ateşlediği mermilerin oluşturulması, fırlatılması ve ekrana çizdirilmesi işlemleri bu dosya üzerinden yürütülür.
- **meteor.c:** Dinamik düşman nesneleri olan meteorların yönetildiği dosyadır. Meteorların rastgele boyut ve konumlarda üretilmesi, oyun alanına belirli açılarla fırlatılması, koordinatlarının anlık olarak güncellenmesi ve render işlemleri bu modülde toplanmıştır.
- **carpisma.c:** Oyundaki fiziksel temasların hesaplandığı matematiksel modüldür. Bu dosya; mermilerin meteorlara isabet etmesini ve meteorların doğrudan uzay gemisine çarpmasıyla oyuncunun hasar almasını denetler. Parçalanma ve can eksilme tetikleyicileri bu fonksiyonlar üzerinden çalışır.
- **arayuz.c:** Oyunun görsel arayüzünden ve menülerinden sorumlu multimedya dosyasıdır. Giriş, duraklatma ve oyun sonu ekranlarının çizdirilmesi işlemlerini üstlenir. Can barı, skor ve meteor canı gibi dinamik verileri ekrana yansıtmanın yanı sıra; en yüksek skorun bilgisayarda kalıcı olması için "rekor.txt" dosyasından veri okunması ve yazılması işlemlerini yürütür. Ayrıca kalkan ve üçlü mermi gibi güçlendirmelerin görsel sayaçları bu modül tarafından yönetilir.

5. Algoritmalar ve Oyun Mekaniği

Projede yer alan nesne etkileşimleri ve fizik kuralları, C dili üzerinden kurgulanmıştır. Oyunun temel yapısını oluşturan algoritmalar şu şekildedir:

- **Gemi Hareketi ve İvmelenme Algoritması:** Geminin hareketi, uzay boşluğundaki sürtünmesiz ortamı simüle edecek şekilde vektörel olarak tasarlanmıştır. Geminin yön açısı radyana çevrilerek, sinüs ve kosinüs fonksiyonlarıyla X ve Y eksenlerindeki hız değerlerine eklenir. İleri yönlü itki kesildiğinde geminin anında durmaması için anlık hız değerleri her karede 0.99 ile çarpılarak yavaşlatılır. Frenleme mekanizması kullanıldığında ise bu katsayı 0.95'e düşürülerek hızlı duruş sağlanır.
- **Mermi Ateşleme Sistemi:** Mermi ateşleme mekanizmasında bekleme süresi algoritması kullanılmıştır. Oyuncu ateş tuşuna bastığında bir sayaç başlatılır ve bu sayaç sıfırlanana kadar yeni mermi üretilmez. Üçlü atış yeteneği aktif olduğunda ise, tek bir mermi yerine ana açığa -15, 0 ve +15 derecelik sapmalar eklenerek aynı anda 3 farklı vektörde mermi objesi oluşturulur.
- **Meteor Üretimi:** Meteorlar ekranın ortasında birden belirlemek yerine, oyun alanının dışındaki 4 farklı kenarda rastgele üretilir. Üretilen her meteorun rotası, ekranın tam merkezine doğru yönlendirilir ve hız vektörleri bu merkez noktasına olan uzaklığa göre hesaplanır.
- **Meteor Hareketi ve Ekran Sarma:** Meteorlar ve uzay gemisi ekran sınırlarının dışına çıktığında yok olmak yerine ekran sarma algoritması ile ekranın diğer tarafından giriş yapar. Örneğin X ekseninde ekran genişliğini aşan bir obje, X ekseninin sıfır noktasına ışınlanarak oyun alanının sonsuz bir uzay boşluğu gibi hissettirmesi sağlanır.

- **Çarpışma Algoritması:** Çarpışma kontrollerinde objelerin dikdörtgen sınırlarının kesişip kesişmediği denetlenir. Eğer bir meteor mermi ile kesişirse, meteorun boyutuna göre parçalanma algoritması devreye girer. Ana meteor bellekten silinirken, bulunduğu koordinatlarda zıt yönlerde hareket eden iki yeni "yavru meteor" üretilir. Meteor gemiye çarptığında ise merkez noktaları karşılaştırılarak gemiye çarpışmanın yönüne doğru bir savrulma ivmesi uygulanır.
- **Zorluk Sistemi:** Oyunun zorluk eğrisi oyuncunun skoruna bağlanmıştır. Skor her 500 puana ulaştığında ekranda görsel bir uyarı verilir ve oyunun parametreleri zorlaştırılır. Bu sistemde doğan yeni meteorların başlangıç canları artırılırken, oyuncuya avantaj sağlayan güçlendirme meteorlarının çıkma ihtimali matematiksel olarak düşürülür.
- **Oyun Döngüsü:** Oyunun ana iskeleti Durum Makinesi algoritmasıyla kurulmuştur. Ana "while" döngüsü içerisinde oyun; GİRİŞ, OYUN, DURAKLATMA ve OYUN SONU olmak üzere farklı durumlara bölünmüştür. Döngü her saniye klavye/fare girdilerini okur, fizik hesaplamalarını yapar ve grafikleri günceller. Bu yapı, kodun karmaşıklığını önleyerek menüler arası geçişin pürüzsüz olmasını sağlar.

6. Karşılaşılan Zorluklar

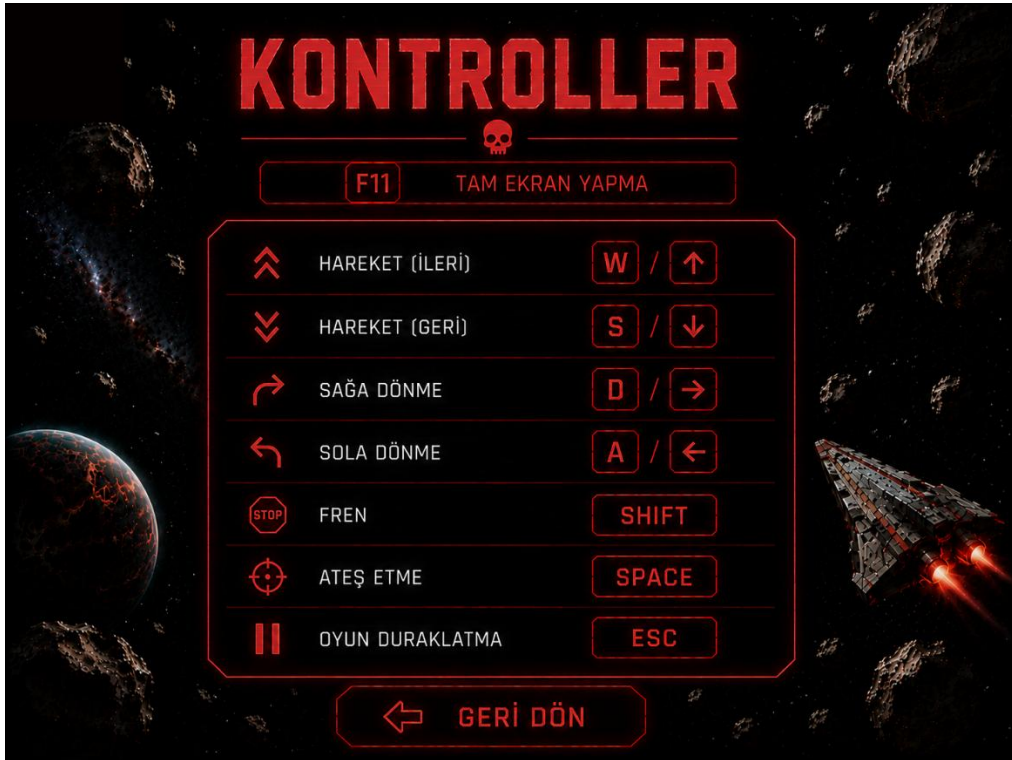
Oyun motoru kullanılmadan, tamamen C dili ve SDL2 kütüphanesiyle sıfırdan geliştirilen bu projede çeşitli yapısal ve görsel zorluklarla karşılaşmış, bu problemlere yönelik çözümler üretilmiştir:

- **Veri Yapılarının Yönetimi ve Pointer Kullanımı:** Oyun içindeki uzay gemisi, meteorlar ve mermiler gibi çoklu özellik barındıran nesneler "struct" yapılarıyla tanımlanmıştır. Başlangıçta bu büyük veri yapılarının fonksiyonlar arasında doğrudan kopyalanarak aktarılmaya çalışılması kod karmaşasına ve performans kayıplarına yol açmıştır. Bu zorluk, struct yapılarının fonksiyonlara pointer aracılığıyla referans olarak gönderilmesiyle aşılmış ve kodun bellek kullanımı optimize edilmiştir.
- **Meteorlara Can Özelliğinin Entegre Edilmesi:** Başlangıç aşamasında tek mermiyle yok olan meteorlara bir can sistemi eklenmesi çarpışma algoritmasını zorlaştırmıştır. Merminin her temasında meteorun anında yok olmasını engelleyip, öncelikle can değerinin düşürülmesi; can değeri sıfırlandığında ise parçalanma mekanizmasının tetiklenmesi için çarpışma motorunda if-else blokları kurgulanmıştır.
- **Parçalanma Esnasında Oluşan Grafikselleşmeler:** Büyük bir meteor vurulduğunda ekrandan silinip yerine iki yeni yavru meteorun çizdirilmesi esnasında, oyunun render döngüsündeki çok kısa süreli senkronizasyon kayıpları yüzünden ekranda parlamalar ve anlık görsel hatalar meydana gelmiştir. Bu sorun, nesnelerin bellekten silinme ve yeniden oluşturulma adımlarının ana çizim döngüsüyle tam senkronize hale getirilmesiyle çözülmüştür.
- **Tam Ekran Geçişleri ve Pencere Boyutlandırma Çakışması:** Oyun penceresi oluşturulurken boyutlandırılabilirlik parametresi aktif edildiğinde; sağ üstteki standart büyütme butonu ile klavyedeki F11 tam ekran kısayolunun senkronize çalışmadığı tespit edilmiştir. Büyütme butonu kullanıldıktan sonra F11 ile pencereyi küçültmek için tuşa art arda iki kez basılması gerekiyordu. İşletim sisteminin standart pencere büyütme işlemi ile SDL2'nin tam ekran motorunun arka planda farklı durumlar oluşturduğu anlaşıldığından; standart büyütme özelliği koddan çıkarılmış ve tam ekran geçişleri yalnızca F11 tuşuna atanarak bu senkronizasyon sorunu tamamen çözülmüştür.

7. Ekran Görüntüleri



Şekil 1: Asteroids Oyunu giriş ekranı arayüzü



Şekil 2: Asteroids Oyunu kontroller menüsü arayüzü



Şekil 3: Asteroids Oyunu oyun içi ekran görüntüsü

8. Sonuç

Bu projede C programlama dili ve SDL2 multimedya kütüphanesi kullanılarak gerçek zamanlı çalışan 2D bir uzay oyunu sıfırdan geliştirilmiştir. Fizik, çarpışma ve bellek yönetimi gibi arka plan sistemleri matematiksel algoritmalar ve kodlama teknikleriyle bizzat tasarlanmıştır.

Geliştirme süreci boyunca; pointer kullanımı, veri yapılarının fonksiyonlar arası aktarımı ve çoklu nesnelerin sabit boyutlu diziler üzerinden durum kontrolüyle güvenli bir şekilde yönetilmesi gibi C dilinin kritik özellikleri tecrübe edilmiştir. Aynı zamanda projeyi tek bir dosyaya yığmak yerine bağımsız dosyalara bölerek modüler programlama ve yazılım mimarisi kurma becerisi kazanılmıştır.

Sonuç olarak, sadece ekranda çalışan eğlenceli bir oyun ortaya konmamış; aynı zamanda bir oyun motorunun arka planda nasıl çalıştığına dair oldukça fazla bilgi edinilmiştir.

9. Kaynakça

1. SDL2 Resmi Dokümantasyonu. Erişim adresi: <https://wiki.libsdl.org/>
2. C Programlama Dili Standart Kütüphane Referansları. Erişim adresi: <https://en.cppreference.com/w/c>
3. Kenney - Uzay gemisi ve arayüz gibi oyun içi telifsiz 2D grafik varlıkları (Sprites). Erişim adresi: <https://www.kenney.nl/>
4. Freesound ve Pixabay - Arka plan müzikleri, patlama, lazer ses efektleri ve multimedya içerikleri. Erişim adresleri: <https://freesound.org/> , <https://pixabay.com/>
5. Gemini ve ChatGPT- Geliştirme sürecinde kod optimizasyonu, hata ayıklama ve meteor mantığının/tasarımlarının oluşturulması aşamalarında yapay zeka asistan desteği.